

LPE in InTune Windows Agent MSI

 medium.com/@offsecdeer/lpe-in-intune-windows-agent-msi-8b2a37edee61

Giulio Pierantoni

March 25, 2025

This blog is about a little LPE vulnerability I found while testing a Windows 11 VM serviced by InTune. Part of my local recon routine consists in looking at the cached MSI installers as they may allow unprivileged users to launch repair operations. Program repairs are interesting from an attacker's point of view because they are forced to run as SYSTEM, since they need elevated privileges to fix a broken install. This means unprivileged users can spawn SYSTEM processes at will with MSI installers, either by clicking on them or through *msiexec.exe*.

It is here that many developers accidentally introduce vulnerabilities in their installers, because if any of these operations can be hijacked by an unprivileged user they will gain the ability to execute actions as SYSTEM. One common exploitable behavior is running an external terminal application as SYSTEM, which is the case covered here.

Recon

Windows stores the MSI of installed programs in *C:\Windows\Installer*, giving them a very unhelpful random name. The same installer will hold a different name on two different hosts, and on top of that existing installers can get automatically renamed. If we add the *Subject* column to Explorer we can view the program they belong to.

> This PC > Local Disk (C:) > Windows > Installer			
	Name ^	Subject	Date modified
★	 2862256b	Microsoft Windows Desktop Runtime - 8.0.6 (x86)	6/26/2024 2:53 PM
★	 2862256f	Microsoft Windows Desktop Runtime - 8.0.6 (x64)	6/26/2024 2:54 PM
★	 2862257b	Microsoft .NET Host FX Resolver - 8.0.6 (x86)	6/26/2024 2:52 PM
★	 2862257f	Microsoft .NET Host FX Resolver - 8.0.6 (x64)	6/26/2024 2:52 PM
★	 28622462	vs_vswebprotocolselectormsi	6/26/2024 2:37 PM
	 28622466	vs_githubprotocolhandlermsi	6/26/2024 2:37 PM
	 28622472	vs_minshellmsires	6/26/2024 2:42 PM
	 28622476	vs_minshellsharedmsi	6/26/2024 2:42 PM
	 28622482	vs_filehandler_amd64	6/26/2024 2:42 PM

[msi-search](#) is a little utility that allows to enumerate these installers through WMI, useful if we don't have access to a GUI.

```

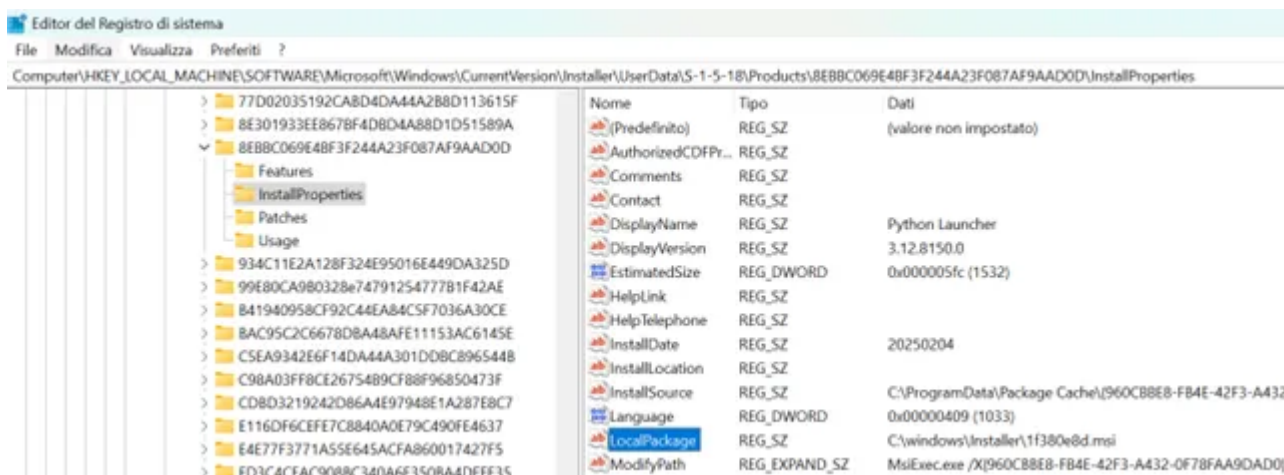
Windows PowerShell
PS C:\Users\Giulio Pierantoni\Desktop> .\msi_search.ps1
-----
File: C:\Windows\Installer\11d2a.msi
Manufacturer: Fortinet Technologies Inc
ProductName: FortiClient VPN
ProductVersion: 7.0.10.0538
-----
File: C:\Windows\Installer\130bbf.msi
Manufacturer: Poly, Inc.
ProductName: Poly Lens
ProductVersion: 1.1.28.5852
-----
File: C:\Windows\Installer\1320f0a6.msi
Manufacturer: dotPDN LLC
ProductName: paint.net
ProductVersion: 5.0.13
-----
File: C:\Windows\Installer\14182e.msi
Manufacturer: HP Inc.
ProductName: HP Sure Run Module
ProductVersion: 5.0.5.59
-----

```

If we can't query WMI we can always query the registry, which is where Windows keeps track of each user's installed programs:

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Installer\UserData\<UserSID>\Products

Every registered application has a folder named after its GUID, and in each one we can find the path of the cached installer under *InstallProperties\LocalPackage*.



I wrote a little [PowerShell script](#) to automatically list installers found in the registry.

```
Windows PowerShell
PS C:\Users\Giulio Pierantoni\desktop> .\EnumMSI.ps1
-----
Program: Office 16 Click-to-Run Extensibility Component
MSI: C:\windows\Installer\3bc04ed.msi
Version: 16.0.18025.20104
-----
Program: Office 16 Click-to-Run Localization Component
MSI: C:\windows\Installer\3bc04f1.msi
Version: 16.0.18025.20104
-----
Program: Office 16 Click-to-Run Licensing Component
MSI: C:\windows\Installer\3bc04f7.msi
Version: 16.0.18025.20104
-----
Program: paint.net
MSI: C:\windows\Installer\1320f0a6.msi
Version: 5.0.13
-----
```

While going through the MSI files on my target I noticed a cmd window would quickly flash on screen when launching *InTune Windows Agent Package*, so I looked deeper.

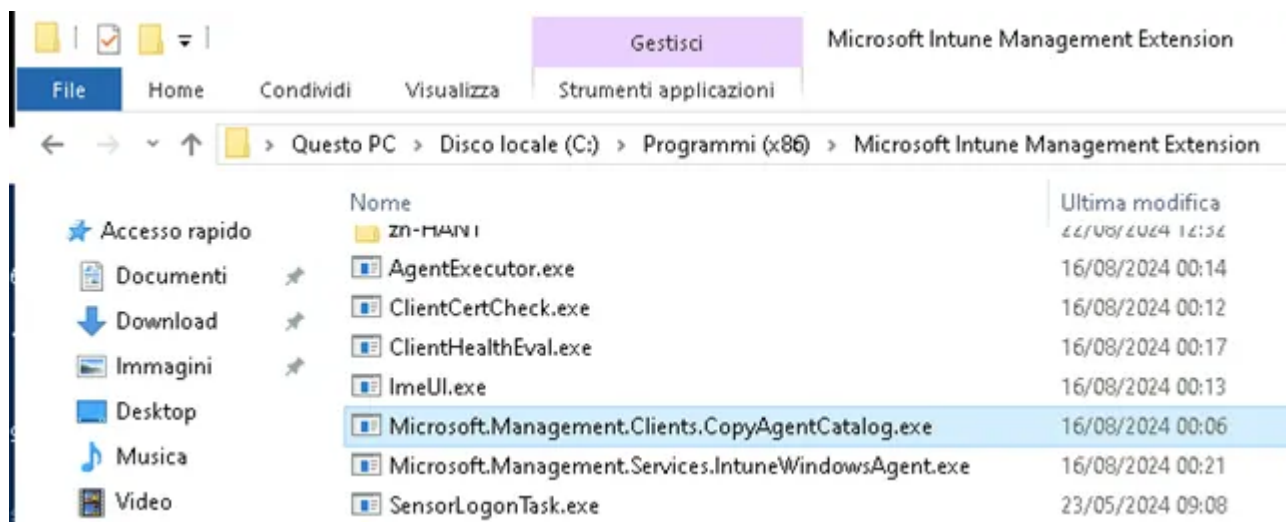
Questo PC > Disco locale (C:) > Windows > Installer >	
Nome	Oggetto
199359.msi	Visual C++ 2017 x86 Additional Runtime
199361.msi	Visual C++ 2013 x64 Additional Runtime
199559.msp	
b7e55.msi	IntuneWindowsAgent Package
ef7e5b1.msi	EPMAgent Package

One way to analyze MSI installers is through Microsoft's Orca tool, obtainable from the Windows SDK. An MSI is essentially a relational database with several tables that dictate the behavior of the installation process, and Orca allows us to see their content.

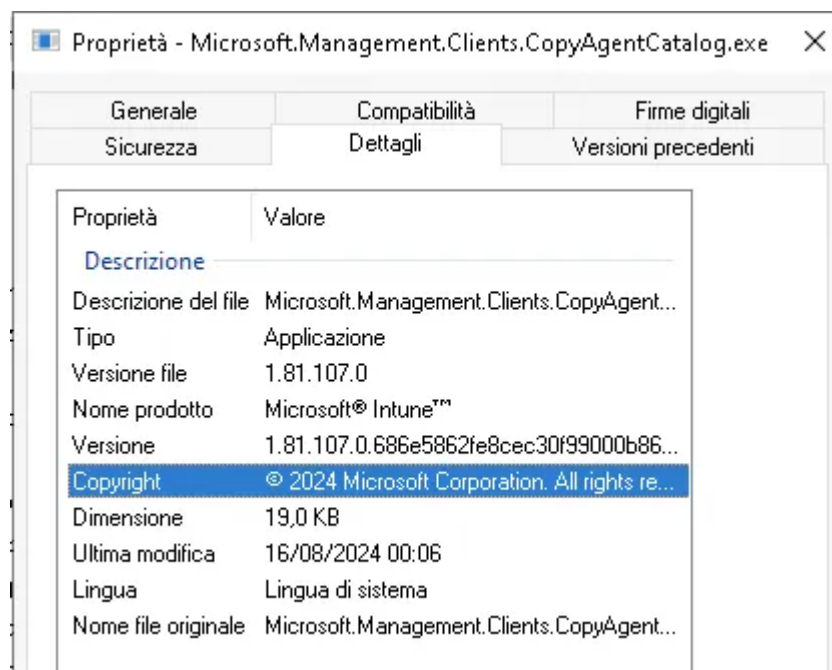
External applications that need to be run during installation/repair/removal are defined in the *CustomAction* table, and it's here that we can find the full path of the flashing window.

b7e55.msi - Orca				
File Edit Tables Transform Tools View Help				
Tables	Action	T...	Source	Target
AdminExecuteSeq...	CreateRegistryKeyAn...	3073	IntuneWindowsAge...	CreateRegistryKeyAndValues
AdminUISequenc...	RemoveRegistryKey...	3137	IntuneWindowsAge...	RemoveRegistryKeyAndValues
AdvExecuteSeque...	CreateClientHealthS...	3073	IntuneWindowsAge...	CreateClientHealthScheduledTask
AppSearch	RemoveClientHealth...	3137	IntuneWindowsAge...	RemoveClientHealthScheduledTask
Binary	CreateClientCertChe...	3073	IntuneWindowsAge...	CreateClientCertCheckScheduledTask
Component	RemoveClientCertCh...	3137	IntuneWindowsAge...	RemoveClientCertCheckScheduledTask
CustomAction	ConfigureInstallation...	3137	IntuneWindowsAge...	ConfigureInstallationFolderPermission
Directory	UpgradeCustomActi...	3137	IntuneWindowsAge...	UpgradeCustomAction
DrLocator	SetServiceSettingsAc...	3137	IntuneWindowsAge...	SetServiceSettingsAction
Feature	StopProcessesDuring...	65	IntuneWindowsAge...	StopProcessesDuringUpgrade
FeatureComponents	RegisterAUMID	2113	IntuneWindowsAge...	RegisterAUMID
File	UnregisterAUMID	2113	IntuneWindowsAge...	UnregisterAUMID
InstallExecuteSequ...	CopyCatalog	3106	INSTALLLOCATION	"[INSTALLLOCATION]Microsoft.Management.Clients.CopyAgentCatalog.exe"
InstallUISequenc...	InstallEtwProvider	3106	INSTALLLOCATION	"[WindowsFolder]system32\wevtutil.exe" im "[INSTALLLOCATION]SideCarETWS

The *CopyCatalog* action launches an external program at *C:\Program Files (x86)\Microsoft Intune Management Extension\Microsoft.Management.Clients.CopyAgentCatalog.exe*.



This executable is also signed by Microsoft.



The best way to find out what is going on during the execution of an MSI is through Sysinternals Procmon. Here we notice *msiexec.exe*, the program used to run MSI files under the hood, launching a process for the aforementioned binary. This program is running in the context of SYSTEM.



When analyzing MSI files it's also important to check the DACL of each program ran during the repair process, including the existence of the DLL they load. It's not uncommon to find misconfigured DACL that could allow an attacker to replace the target binary with a malicious one, or missing DLLs that can be implanted and executed with DLL hijacking.

Either way we now need to find a way to interact with the flashing window.



Exploitation

The easiest method consists in slowing down the system, for example by spawning hundreds of windows in a simple loop right after launching the MSI. We can spawn them minimized so they don't prevent us from interacting with our target, and this should leave the cmd window open just long enough for us to catch it. This PowerShell one-liner should do the job:

```
for ($i = 0; $i -lt 200; $i++) { Start-Process -WindowStyle Minimized powershell.exe }
```

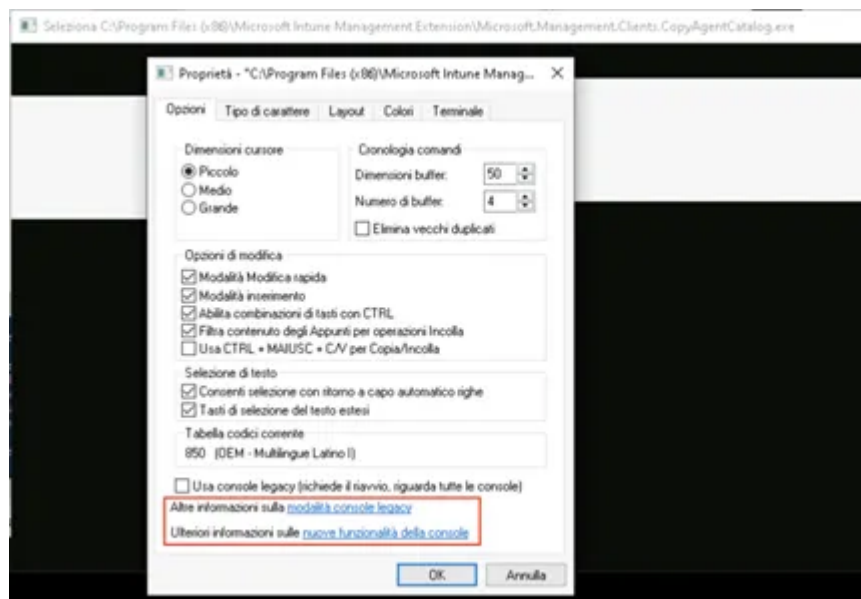
A screenshot of a PowerShell window. The title bar reads "Senza nome1.ps1 X". The command prompt shows the execution of the PowerShell one-liner: `1 for ($i = 0; $i -lt 200; $i++) { Start-Process -WindowStyle Minimized powershell.exe }`. Below the command, the prompt shows the execution progress: `PS C:\Users\Test7Layers> for ($i = 0; $i -lt 200; $i++) { Start-Process -WindowStyle Minimized powershell.exe }` and `PS C:\Users\Test7Layers>`.

This turned out to be an effective and reliable method on the VM I was on because it didn't have a lot of resources, but computers with higher amounts of memory aren't going to be bothered as much.

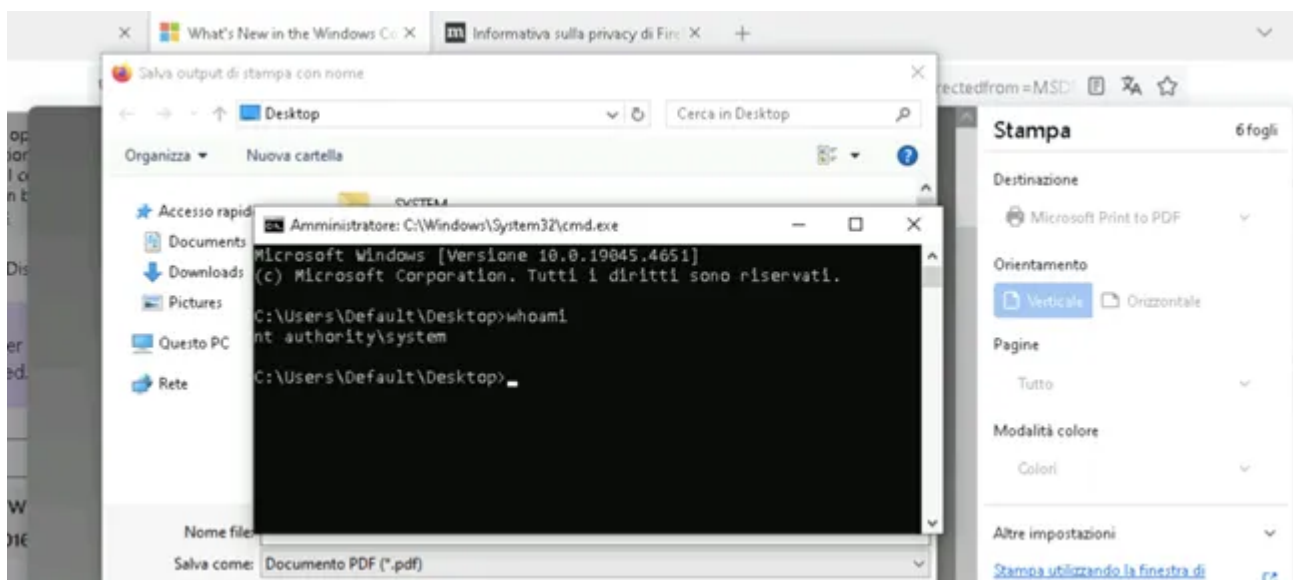
With the computer slowed down we should be able to select some text in the window body, this way it should not close even if the program is done running, then right click on the title bar and select Properties.



The Properties dialog has two blue links at the bottom, clicking on them triggers the execution of a web browser running under SYSTEM, since it is spawned by another SYSTEM process.



From the browser we can open the Print menu and select Microsoft Print to PDF as the printer, so that a Save File dialog appears on screen. Finally, if we click on the path bar at the top, we can type `C:\Windows\System32\cmd.exe`, press enter, and we'll have a cmd window running as SYSTEM. From here we can add ourselves to the local admin group, establish persistence, or do whatever we fancy.



The issue was reported to Microsoft, who fixed it towards the end of October 2024. If your Azure tenant uses InTune to service Windows VMs you should check if your hosts are still affected.

Issues like this show how application developers need to pay close attention to how their installation and repair process is structured: repairs can be triggered by any unprivileged user and can lead to the execution of arbitrary commands as SYSTEM if any of these privileged actions can be hijacked, such as spawning a new console application as SYSTEM or attempting to load a missing DLL from a writable location.

I also hope pentesters start paying more attention to cached MSI files, as they can reveal themselves to be an unexpected ally.